

Modelagem

Arthur Henrique Cavalcante Rosa

2026-09-06

Contents

1	Carga das Bases Processadas	1
2	Treinamento da Regressão Logística	2
3	Avaliação no Teste Out-of-Time	2
4	Visualização Gráfica Acumulada das Métricas	3
4.1	Curva ROC	3
4.2	Curva KS (Kolmogorov-Smirnov)	4
5	Tabela de Decis de Score e Calibração	7
6	Segmentação de Ratings via Árvore de Decisão	8
7	Política de Utilização do Modelo	9
7.1	Ratings de Crédito	9
7.2	Sugestão de Direcionais de Negócio	9
8	Salvamento da Base Final Escorada	9
9	Serialização do Modelo Final	9

1 Carga das Bases Processadas

```
# Carregamento do dataset
dados_treino <- read_csv("../data/trusted/feature_store_treino.csv")
dados_teste <- read_csv("../data/trusted/feature_store_inferencia.csv")

names(dados_treino) <- make.names(names(dados_treino))
names(dados_teste) <- make.names(names(dados_teste))

# Shape dos dados
cat("Linhas Treino:", nrow(dados_treino), "| Colunas:", ncol(dados_treino), "\n\n")

## Linhas Treino: 23614 | Colunas: 21

cat("Linhas Teste:", nrow(dados_teste), "| Colunas:", ncol(dados_teste), "\n\n")

## Linhas Teste: 5695 | Colunas: 21

# Remove colunas de metadados mantendo apenas features explicativas e target
colunas_excluir <- c("issue_d", "id")
```

```
dados_treino_modelo <- dados_treino |> select(-any_of(colunas_excluir))
dados_teste_modelo <- dados_teste |> select(-any_of(colunas_excluir))
```

2 Treinamento da Regressão Logística

```
# Alinhamento estrito de colunas (mesma ordem e nomes exatos do treino)
dados_teste_modelo <- dados_teste_modelo |>
  select(all_of(names(dados_treino_modelo)))

# Treinamento da Regressão Logística no período Jan-Set
modelo_glm <- glm(
  fl_default ~ .,
  data = dados_treino_modelo,
  family = binomial(link = "logit")
)

# Resumo dos coeficientes estatisticamente relevantes (p-valor < 0.05)
summary(modelo_glm)$coefficients |>
  as.data.frame() |>
  rownames_to_column("variavel") |>
  filter(`Pr(>|z|)` < 0.05) |>
  arrange(`Pr(>|z|)`)
```

##	variavel	Estimate	Std. Error	z value	Pr(> z)
## 1	int_rate	1.027576e-01	3.772860e-03	27.235989	2.435500e-163
## 2	addr_state_encoded	5.947322e+00	5.834811e-01	10.192827	2.134647e-24
## 3	loan_amnt	2.384209e-05	2.339712e-06	10.190181	2.193556e-24
## 4	fico_score	-6.598122e-03	6.591425e-04	-10.010160	1.375298e-23
## 5	acc_open_past_24mths	4.489548e-02	6.109371e-03	7.348626	2.002546e-13
## 6	dti	1.290210e-02	2.153221e-03	5.991999	2.072778e-09
## 7	home_ownership_RENT	2.374299e-01	4.222562e-02	5.622888	1.877911e-08
## 8	tot_cur_bal	-9.105462e-07	1.770842e-07	-5.141883	2.719990e-07
## 9	annual_inc	-2.576204e-06	5.945350e-07	-4.333142	1.469964e-05
## 10	mo_sin_old_rev_tl_op	-4.528827e-04	1.969023e-04	-2.300038	2.144608e-02

3 Avaliação no Teste Out-of-Time

```
# Geração dos scores/probabilidades de default
pred_prob <- predict(modelo_glm, newdata = dados_teste_modelo, type = "response")

# Curva ROC e AUC
curva_roc <- roc(dados_teste_modelo$fl_default, pred_prob, quiet = TRUE)
auc_val <- as.numeric(auc(curva_roc))

# Estatística Kolmogorov-Smirnov (KS)
df_ks <- tibble(
  alvo = dados_teste_modelo$fl_default,
  score = pred_prob
)

ks_val <- max(
```

```

abs(
  ecdf(df_ks$score[df_ks$alvo == 1])(df_ks$score) -
  ecdf(df_ks$score[df_ks$alvo == 0])(df_ks$score)
)
)

tibble(
  Métrica = c("AUC-ROC", "KS"),
  Valor = c(round(auc_val, 4), round(ks_val, 4))
)

```

```

## # A tibble: 2 x 2
##   Métrica Valor
##   <chr>   <dbl>
## 1 AUC-ROC 0.704
## 2 KS     0.306

```

4 Visualização Gráfica Acumulada das Métricas

4.1 Curva ROC

```

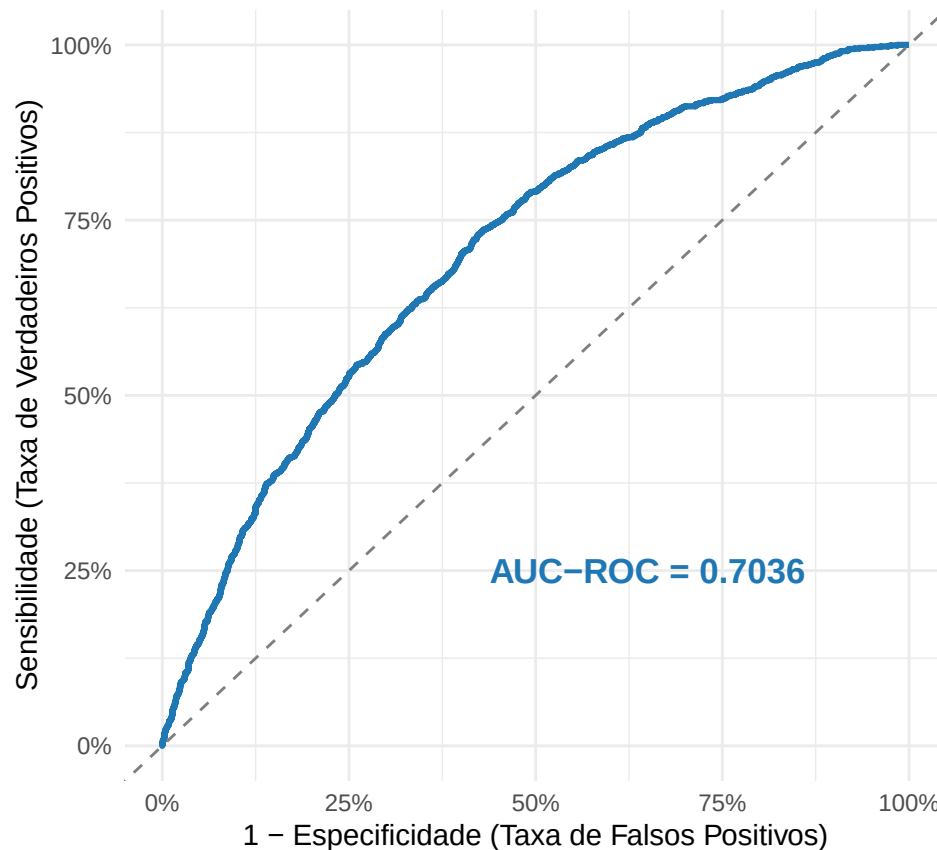
# Plot da Curva ROC
curva_roc <- roc(dados_teste_modelo$fl_default, pred_prob, quiet = TRUE)
df_roc <- tibble(
  especificidade = curva_roc$specificities,
  sensibilidade = curva_roc$sensitivities
)

ggplot(df_roc, aes(x = 1 - especificidade, y = sensibilidade)) +
  geom_line(color = "#1f77b4", linewidth = 1.1) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "gray50") +
  annotate(
    "text", x = 0.65, y = 0.25,
    label = paste0("AUC-ROC = ", round(auc(curva_roc), 4)),
    size = 4.5, fontface = "bold", color = "#1f77b4"
  ) +
  scale_x_continuous(labels = scales::percent_format()) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    title = "Curva ROC - Teste Out-of-Time",
    subtitle = "Capacidade de ordenação global de risco do modelo",
    x = "1 - Especificidade (Taxa de Falsos Positivos)",
    y = "Sensibilidade (Taxa de Verdadeiros Positivos)"
  ) +
  theme_minimal()

```

Curva ROC – Teste Out-of-Time

Capacidade de ordenação global de risco do modelo



4.2 Curva KS (Kolmogorov-Smirnov)

```
# Plot da Curva KS (Kolmogorov-Smirnov)
df_ks_plot <- tibble(
  alvo = dados_teste_modelo$f1_default,
  score = pred_prob
) |>
  arrange(score) |>
  mutate(
    cum_maus = cumsum(alvo == 1) / sum(alvo == 1),
    cum_bons = cumsum(alvo == 0) / sum(alvo == 0),
    diff_ks = abs(cum_maus - cum_bons)
  )

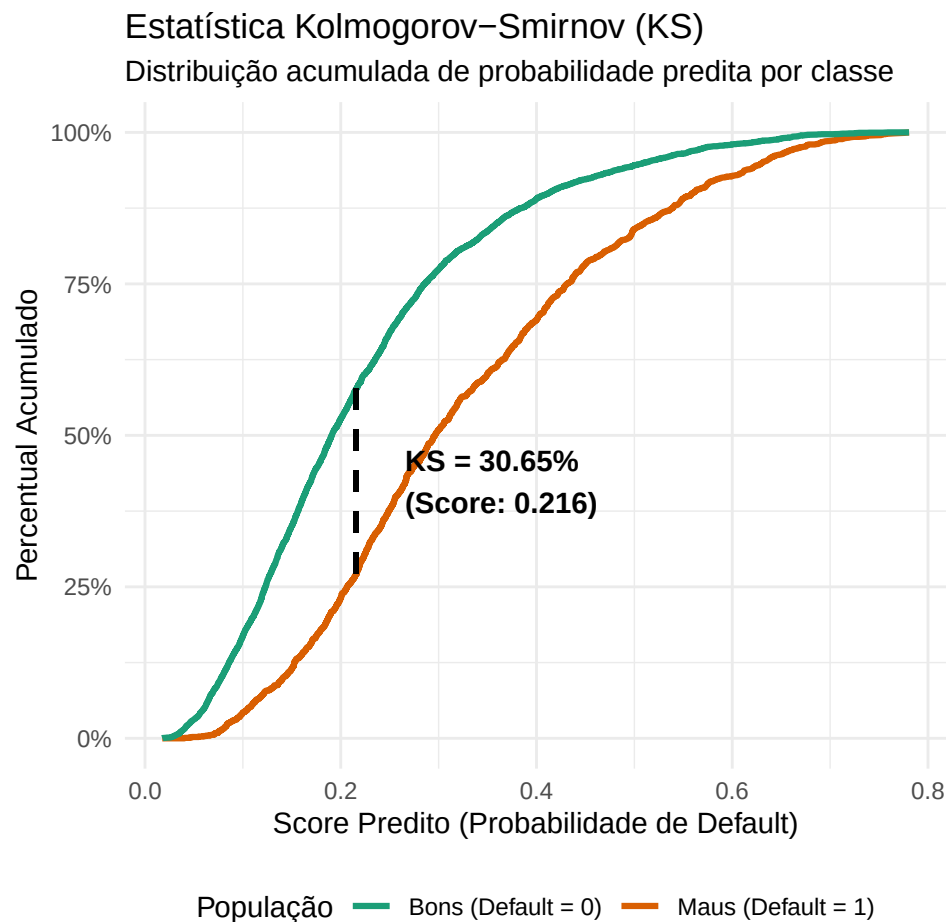
# Ponto de máxima separação
ponto_ks <- df_ks_plot |> slice_max(diff_ks, n = 1, with_ties = FALSE)

ggplot(df_ks_plot, aes(x = score)) +
  geom_line(aes(y = cum_maus, color = "Maus (Default = 1)"), linewidth = 1.1) +
  geom_line(aes(y = cum_bons, color = "Bons (Default = 0)"), linewidth = 1.1) +
  geom_segment(
    aes(x = ponto_ks$score, xend = ponto_ks$score,
```

```

    y = ponto_ks$cum_bons, yend = ponto_ks$cum_maus),
    color = "black", linetype = "dashed", linewidth = 0.9
) +
annotate(
  "text", x = ponto_ks$score + 0.05,
  y = (ponto_ks$cum_maus + ponto_ks$cum_bons) / 2,
  label = paste0("KS = ", round(ponto_ks$diff_ks * 100, 2), "%\n(Score: ", round(ponto_ks$score, 3),
  hjust = 0, fontface = "bold"
) +
scale_color_manual(values = c("Maus (Default = 1)" = "#d95f02", "Bons (Default = 0)" = "#1b9e77")) +
scale_y_continuous(labels = scales::percent_format()) +
labs(
  title = "Estatística Kolmogorov-Smirnov (KS)",
  subtitle = "Distribuição acumulada de probabilidade predita por classe",
  x = "Score Predito (Probabilidade de Default)",
  y = "Percentual Acumulado",
  color = "População"
) +
theme_minimal() +
theme(legend.position = "bottom")

```



por Safras das Métricas

Visualização Gráfica

```

# Função auxiliar para cálculo do KS
calc_ks <- function(alvo, score) {
  df <- tibble(alvo = alvo, score = score)
  max(abs(
    ecdf(df$score[df$alvo == 1])(df$score) -
    ecdf(df$score[df$alvo == 0])(df$score)
  ))
}

# Associa a predição e o target à safra correspondente
df_oot_safra <- tibble(
  issue_d = dados_teste$issue_d,
  alvo    = dados_teste_modelo$fl_default,
  score    = pred_prob
)

# Tabela de performance mensal
metricas_safra <- df_oot_safra |>
  group_by(issue_d) |>
  summarise(
    volume      = n(),
    inadimplencia = mean(alvo),
    auc_roc     = as.numeric(auc(roc(alvo, score, quiet = TRUE))),
    ks          = calc_ks(alvo, score),
    .groups     = "drop"
  ) |>
  arrange(issue_d)

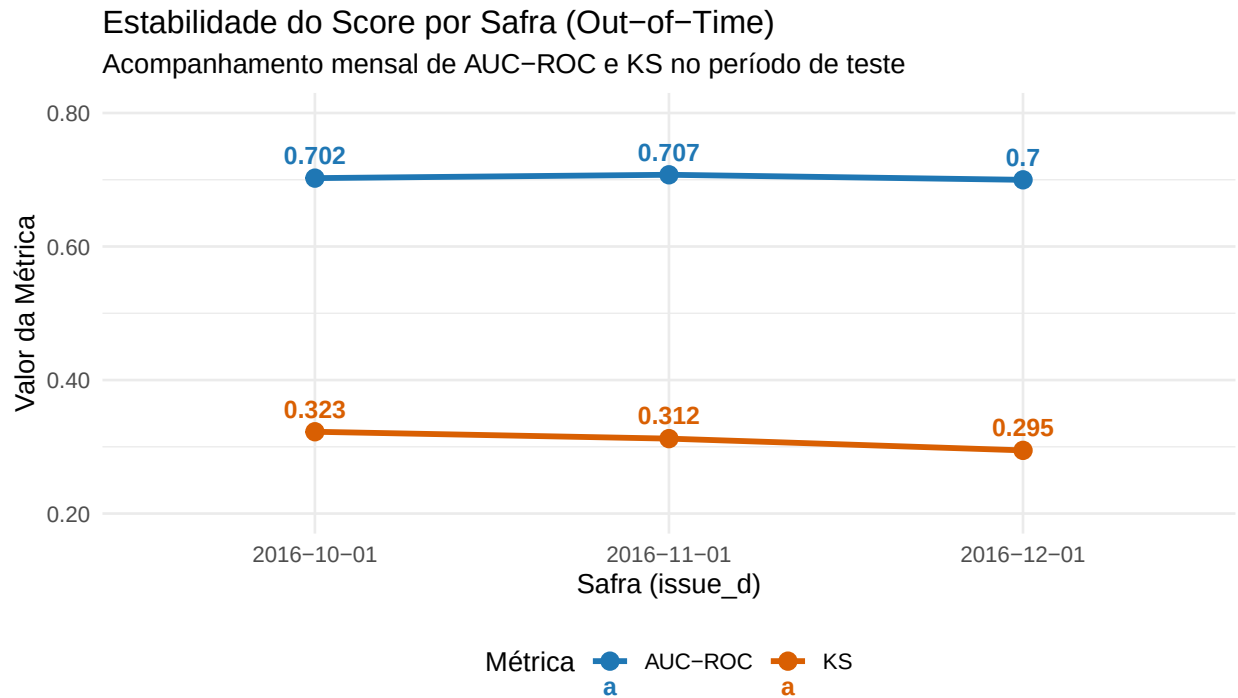
metricas_safra

## # A tibble: 3 x 5
##   issue_d    volume inadimplencia auc_roc    ks
##   <chr>      <int>      <dbl>    <dbl> <dbl>
## 1 2016-10-01   1916          0.229    0.702 0.323
## 2 2016-11-01   1886          0.232    0.707 0.312
## 3 2016-12-01   1893          0.255    0.700 0.295

metricas_safra |>
  pivot_longer(cols = c(auc_roc, ks), names_to = "metrica", values_to = "valor") |>
  mutate(metrica = if_else(metrica == "auc_roc", "AUC-ROC", "KS")) |>
  ggplot(aes(x = as.factor(issue_d), y = valor, group = metrica, color = metrica)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 3) +
  geom_text(aes(label = round(valor, 3)), vjust = -0.8, fontface = "bold", size = 3.5) +
  scale_y_continuous(limits = c(0.2, 0.8), labels = scales::number_format(accuracy = 0.01)) +
  scale_color_manual(values = c("AUC-ROC" = "#1f77b4", "KS" = "#d95f02")) +
  labs(
    title = "Estabilidade do Score por Safra (Out-of-Time)",
    subtitle = "Acompanhamento mensal de AUC-ROC e KS no período de teste",
    x = "Safra (issue_d)",
    y = "Valor da Métrica",
    color = "Métrica"
  ) +
  theme_minimal() +

```

```
theme(legend.position = "bottom")
```



5 Tabela de Decis de Score e Calibração

```
# Construção dos decis de risco ordenados
tabela_decis <- df_oot_safra |>
  mutate(decil = ntile(score, 10)) |>
  group_by(decil) |>
  summarise(
    qtd_clientes = n(),
    score_min = min(score),
    score_max = max(score),
    qtd_default = sum(alvo),
    taxa_default = mean(alvo),
    .groups = "drop"
  ) |>
  arrange(desc(decil)) |>
  mutate(
    perc_default_acum = cumsum(qtd_default) / sum(qtd_default),
    perc_base_acum = cumsum(qtd_clientes) / sum(qtd_clientes)
  )
```

tabela_decis

```
## # A tibble: 10 x 8
##   decil qtd_clientes score_min score_max qtd_default taxa_default
##   <int>      <int>      <dbl>    <dbl>      <int>      <dbl>
## 1     10         569     0.469     0.781         271     0.476
## 2      9         569     0.364     0.468         241     0.424
## 3      8         569     0.295     0.364         173     0.304
```

```
## 4      7      569    0.250    0.295      161    0.283
## 5      6      569    0.215    0.250      149    0.262
## 6      5      570    0.183    0.215      115    0.202
## 7      4      570    0.152    0.183       88    0.154
## 8      3      570    0.121    0.152       62    0.109
## 9      2      570    0.0856   0.121       65    0.114
## 10     1      570    0.0177   0.0855       34    0.0596
## # i 2 more variables: perc_default_acum <dbl>, perc_base_acum <dbl>
```

6 Segmentação de Ratings via Árvore de Decisão

```
library(rpart)
# Ajusta árvore unidimensional para maximizar separação de default
# maxdepth = 3 limita em até 8 nós terminais (ratings)
# minbucket garante que cada rating tenha representatividade mínima (ex: 5% da base)
tree_ratings <- rpart(
  alvo ~ score,
  data = df_oot_safra,
  method = "anova",
  control = rpart.control(
    maxdepth = 4,
    minbucket = round(0.08 * nrow(df_oot_safra)),
    cp = 0.001
  )
)

# Extrai os splits ótimos e define as fronteiras
cortes_otimos <- sort(unique(tree_ratings$splits[, "index"]))
limites <- c(-Inf, cortes_otimos, Inf)
rotulos_rating <- LETTERS[1:(length(limites) - 1)]

# Tabela final de esteira por Ratings
tabela_ratings_cart <- df_oot_safra |>
  mutate(rating = cut(score, breaks = limites, labels = rotulos_rating, right = TRUE)) |>
  group_by(rating) |>
  summarise(
    qtd_clientes      = n(),
    perc_populacao    = n() / nrow(df_oot_safra),
    score_min         = min(score),
    score_max         = max(score),
    taxa_default_real = mean(alvo),
    .groups           = "drop"
  ) |>
  arrange(rating)

tabela_ratings_cart

## # A tibble: 5 x 6
##   rating qtd_clientes perc_populacao score_min score_max taxa_default_real
##   <fct>      <int>         <dbl>     <dbl>     <dbl>         <dbl>
## 1 A           461         0.0809     0.0177     0.0786         0.0412
## 2 B          975         0.171      0.0787     0.136          0.105
## 3 C         1431         0.251      0.136     0.216          0.172
```

## 4 D	1721	0.302	0.216	0.368	0.282
## 5 E	1107	0.194	0.368	0.781	0.457

7 Política de Utilização do Modelo

7.1 Ratings de Crédito

- **Rating A (Risco Mínimo):** Taxa de *default* esperada de **4%**
 - **Rating B (Risco Baixo):** Taxa de *default* esperada de **10%**
 - **Rating C (Risco Médio):** Taxa de *default* esperada de **17%**
 - **Rating D (Risco Alto):** Taxa de *default* esperada de **28%**
 - **Rating E (Risco Crítico):** Taxa de *default* esperada de **45%**
-

7.2 Sugestão de Direcionais de Negócio

- **Rating A:** Concessão com limites máximos pré-aprovados.
- **Rating B:** Limite pleno compatível com a renda declarada.
- **Rating C:** Limite moderado, priorizando entrada conservadora com esteira de aumento gradual.
- **Rating D:** Limite restritivo e prazos reduzidos.
- **Rating E:** Concessão de limite inviável economicamente (recusa automática para crédito sem garantia).

8 Salvamento da Base Final Escorada

```
library(tidyverse)

# Estruturação da base final com os campos solicitados
base_escorada_oot <- tibble(
  id      = dados_teste$id,
  safra   = dados_teste$issue_d,
  score   = round(pred_prob, 3),
  rating  = cut(pred_prob, breaks = limites, labels = rotulos_rating, right = TRUE)
)

write_csv(base_escorada_oot, "../data/delivery/base_escorada_oot.csv")

cat("Base escorada exportada com sucesso (base_escorada_oot.csv)!\n")
```

```
## Base escorada exportada com sucesso (base_escorada_oot.csv)!
```

9 Serialização do Modelo Final

```
saveRDS(modelo_glm, "../model/modelo_regressao_logistica.rds")

cat("Modelo final serializado com sucesso: modelo_regressao_logistica.rds\n")
```

```
## Modelo final serializado com sucesso: modelo_regressao_logistica.rds
```